arXiv:2601.05214v1 [cs.AI] 8 Jan 2026

Internal Representations as Indicators of Hallucinations in Agent Tool Selection

Kait Healy*, **Bharathi Srinivasan** *, **Visakh Madathil***, **Jing Wu** *

Amazon
410 Terry Avenue North, Seattle, WA 98109, United States

Abstract

Large Language Models (LLMs) have shown remarkable capabilities in tool calling and tool usage, but suffer from hallucinations where they choose incorrect tools, provide malformed parameters and exhibit 'tool bypass' behavior by performing simulations and generating outputs instead of invoking specialized tools or external systems. This undermines the reliability of LLM based agents in production systems as it leads to inconsistent results, and bypasses security and audit controls. Such hallucinations in agent tool selection require early detection and error handling. Unlike existing hallucination detection methods that require multiple forward passes or external validation, we present a computationally efficient framework that detects tool-calling hallucinations in real-time by leveraging LLMs' internal representations during the same forward pass used for generation. We evaluate this approach on reasoning tasks across multiple domains, demonstrating strong detection performance (up to 86.4% accuracy) while maintaining real-time inference capabilities with minimal computational overhead, particularly excelling at detecting parameter-level hallucinations and inappropriate tool selections, critical for reliable agent deployment.

Introduction

Large Language Models (LLMs) have demonstrated remarkable capabilities as autonomous agents, enabling them to interact with external APIs, execute code, and orchestrate complex workflows (Brown et al. 2020; Schick et al. 2023; Qin et al. 2024). However, these models exhibit a critical vulnerability: **tool-calling hallucinations**, where they generate structurally plausible but functionally incorrect tool calls. Unlike textual hallucinations, tool-calling hallucinations manifest as inappropriate tool selection, malformed parameters, incorrect tool chaining, tool bypass behavior, or semantically incorrect function invocations that can lead to system failures or data corruption.

The detection of tool-calling hallucinations presents unique challenges. Tool calls possess rigid structural and semantic constraints—parameters must conform to specific types and must be accurately interpreted from the query to the agent, required arguments cannot be omitted, and function names must exist in the available repertoire. Moreover, the consequences of undetected hallucinations can be severe in computational domains where precision is critical. Incorrect tool usage can also exacerbate security vulnerabilities of agents due to incorrect data access

To address these challenges, we propose a novel approach that leverages the internal representations of LLMs during tool call generation to detect hallucinations in real-time. Our method builds upon recent advances in understanding LLM internal states (Su et al. 2024; Azaria and Mitchell 2023) and extends them to structured tool calling. We introduce an unsupervised training framework that automatically generates labeled data by masking ground-truth tool calls, prompting LLMs to predict appropriate functions, and training lightweight classifiers on the resulting contextualized embeddings.

Our key contributions are:

- We demonstrate that internal representations of LLMs contain discriminative information for detecting tool-calling hallucinations in reasoning tasks.
- We evaluate our approach on tool calling scenarios, showing effective real-time detection capabilities with minimal computational overhead.

Related Work

Hallucination Detection in Large Language Models

Hallucination detection has emerged as a critical challenge in deploying LLMs for real-world applications. Early approaches focused on post-processing methods that analyze generated text for factual inconsistencies (Maynez et al. 2020; Zhou et al. 2020; Ji et al. 2023; Wu et al. 2025b,a; Wu, Hobbs, and Hovakimyan 2023). These methods typically require external knowledge sources or multiple model generations to assess consistency. Recent work has explored uncertainty-based approaches that leverage model confidence signals. Kadavath et al. (2022) demonstrated that LLMs can express uncertainty about their knowledge, while Zhang et al. (2023) proposed enhanced uncertainty estimation methods for hallucination detection. However, these approaches struggle with the discrete nature of tool calling, where traditional uncertainty measures may not capture semantic correctness of API usage. Consistency-based methods represent another major direction. SelfCheckGPT (Manakul, Liusie, and Gales 2023)

*These authors contributed equally.

Copyright © 2026, Association for the Advancement of Artificial Intelligence Workshop.

generates multiple responses and checks for consistency, assuming that hallucinated content will be inconsistent across samples. Non Contradiction Probability (NCP) (Hou et al. 2025) and Semantic Similarity (Kuhn, Arakelyan, and Percha 2023) are two techniques using this principle of consistency to detect hallucinations. After sampling, NCP probabilistically scores whether a response contradicts entries in a curated belief set using information-theoretic scoring or belief tree propagation, while semantic similarity measures the degree of meaning alignment between a model's outputs from multiple samples using cosine similarity or nearest-neighbor score between them. Li et al. (2023a) introduced evaluation frameworks for assessing hallucination detection methods. While effective for free-form text, these approaches face limitations in tool calling scenarios where there may be unique correct solutions, making consistency-based detection less reliable.

Tool-Augmented Language Models

The integration of external tools with language models has gained significant attention as a means to extend LLM capabilities beyond the parametric knowledge acquired from their training data. Toolformer (Schick et al. 2023) pioneered self-supervised learning for tool use, demonstrating that LLMs can learn when and how to call APIs through minimal demonstrations. This work established the foundation for automated tool learning but did not address the detection of incorrect tool usage. ToolLLM (Qin et al. 2024) scaled tool learning to thousands of real-world APIs, introducing comprehensive training frameworks and evaluation benchmarks. Gorilla (Patil et al. 2024) focused specifically on API calling accuracy, achieving strong performance through retrieval-augmented training. ReAct (Yao et al. 2023) proposed reasoning and acting paradigms that interleave tool use with reasoning, while Lu et al. (2023) developed plug-and-play compositional reasoning systems. Despite these advances, existing work primarily focuses on improving tool-use accuracy during training rather than detecting errors during inference. Our work addresses this gap by providing real-time detection capabilities that can identify when models generate inappropriate tool calls.

Internal Representation Analysis

Understanding the internal mechanisms of neural networks has become increasingly important for building reliable AI systems. Azaria and Mitchell (2023) demonstrated that internal states of LLMs indicate the veracity of responses, showing that classifiers trained on hidden representations can detect when models produce false statements. This work provided crucial evidence that internal representations encode semantic properties beyond surface-level text generation. Building on this foundation, Su et al. (2024) developed unsupervised methods for real-time hallucination detection using contextualized embeddings from transformer layers. Their approach eliminated the need for manual annotation by automatically generating training data through entity masking in Wikipedia articles. Our work extends these insights to the structured domain of tool calling, where the challenges and patterns differ significantly from free-form text generation. Recent work has also explored mechanistic interpretability of transformers (Elhage et al. 2021; Wang et al. 2022), providing insights into how these models process and represent information. Meng et al. (2022) investigated knowledge storage and editing in transformers, while Li et al. (2023b) analyzed attention patterns in reasoning tasks. These insights inform our understanding of where tool-calling information might be encoded within transformer representations.

Agent Systems and Reliability

The deployment of LLM-based agents in real-world scenarios has highlighted the need for robust error detection and mitigation strategies. Xi et al. (2023) surveyed the landscape of autonomous agents, identifying reliability as a key challenge for practical deployment. Wang et al. (2023) demonstrated lifelong learning agents but noted the challenges of ensuring consistent performance across diverse environments. Recent work has explored agent evaluation frameworks. Liu et al. (2023) introduced comprehensive benchmarks for agent capabilities, while Xu et al. (2023) proposed evaluation metrics for tool-using agents. However, these evaluation frameworks primarily focus on task completion rather than real-time error detection during agent execution. Safety and alignment in agent systems have also received attention. Hendrycks, Mazeika, and Woodside (2023) discussed risks associated with autonomous AI systems, while Kenton et al. (2021) explored alignment challenges specific to agent deployment. Our work contributes to this area by providing mechanisms for detecting potentially harmful or incorrect agent actions before they impact external systems.

Evaluation Methodologies for tool calling

Benchmarking tool calling capabilities has become increasingly sophisticated as the field has matured. APIBench (Patil et al. 2024) established evaluation protocols for API calling accuracy, focusing on syntactic and semantic correctness of generated calls. ToolBench (Qin et al. 2024) expanded evaluation to include multi-step reasoning and complex tool interactions. Recent work has emphasized the importance of real-world evaluation scenarios. Tang et al. (2023) introduced evaluation datasets based on actual API usage patterns, while Ruan et al. (2023) explored evaluation in dynamic environments where API specifications change over time. These works highlight the challenges of tool calling evaluation but have not addressed real-time error detection during generation.

Problem Formulation

We formalize tool-calling hallucination detection as a binary classification task operating on the internal representations of the last layer of large language models (LLMs) during tool call generation. Let $\mathcal{F} = \{f_1, f_2, \ldots, f_n\}$ denote the set of available functions in an agent's toolkit. Given a user query q and contextual information c, an LLM $\mathcal{M}$ generates a tool call $\tilde{f}(\boldsymbol{a})$ where $\tilde{f} \in \mathcal{F} \cup \{\emptyset\}$ and $\boldsymbol{a}$ represents the function arguments.

We define a tool-calling hallucination as occurring when any of the following conditions hold:

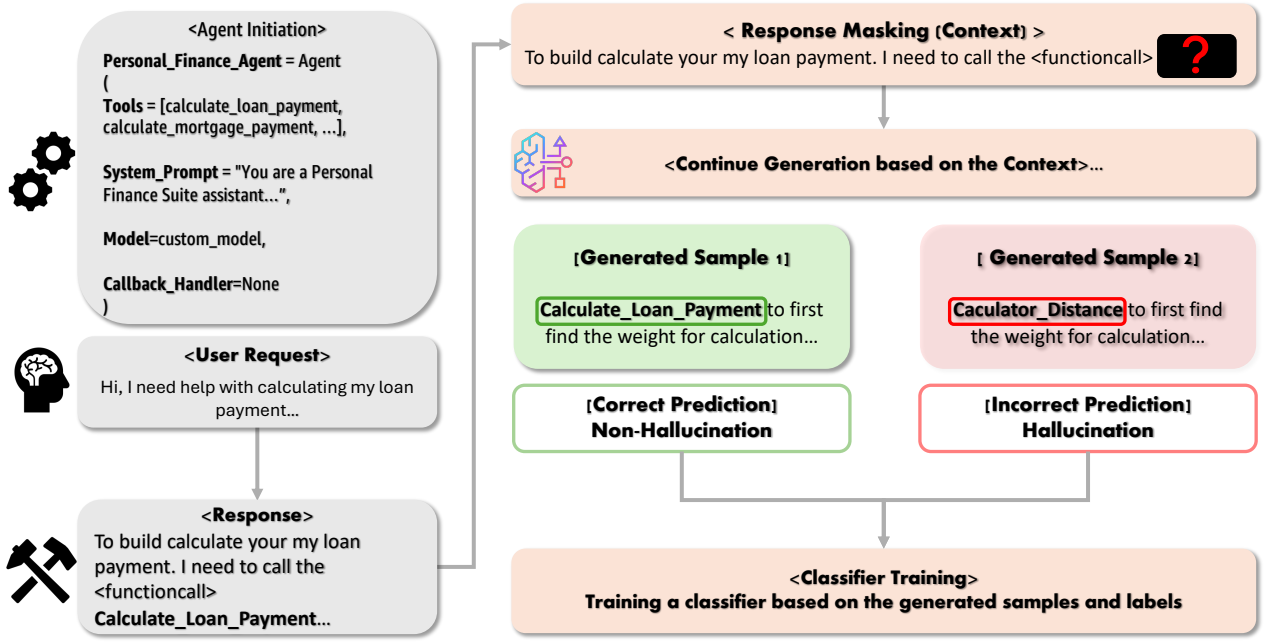

Figure 1: **Unsupervised, single-pass training pipeline for tool-calling hallucination detection.** Given a user query and available tools, the LLM generates candidate responses containing tool calls. We *mask* the tool-call segment to create a context-only prompt, ask the model to predict the call again, and assign labels by agreement with the reference call (non-hallucination vs. hallucination). We then train a lightweight classifier on last-layer representations to distinguish correct from hallucinated calls; at inference, the classifier scores each proposed call in real time and gates execution accordingly.

(1) **Function Selection Error**: $\tilde{f} \notin \mathcal{F}$ (invoking a non-existent function)

(2) **Function Appropriateness Error**: $\tilde{f} \in \mathcal{F}$ but $\tilde{f}$ is semantically inappropriate for query q given context c

(3) **Parameter Error**: $\boldsymbol{a} \notin \mathcal{D}_{\tilde{f}}$, where $\mathcal{D}_{\tilde{f}}$ denotes the valid argument domain for function $\tilde{f}$

(4) **Completeness Error**: Required parameters are missing from $\boldsymbol{a}$

(5) **Tool Bypass Error**: Generating the output without using $\tilde{f} \in \mathcal{F}$

Our objective is to learn a classifier $h_\theta : \mathbb{R}^d \to \{0, 1\}$ that maps the internal representation $\mathbf{z} \in \mathbb{R}^d$ of the LLM at tool call generation time to a binary decision, where 1 indicates a hallucinated tool call and 0 indicates a correct tool call. While we categorize hallucinations into five distinct types for theoretical completeness, our current work focuses on unified binary detection across all error categories, enabling the model to learn general patterns of incorrectness while maintaining practical deployment simplicity.

Method

We detect hallucinations in AI agent tool calling based on the hypothesis that the representations from the final transformer layer during tool call generation contain sufficient information to distinguish between correct and hallucinated calls.

Setup and Notation Let $\mathcal{D} = \{(q_i, c_i, f_i^*, \boldsymbol{a}_i^*, y_i)\}_{i=1}^N$ denote our collected tool calling instances, where q_i represents the user query, c_i the contextual information, f_i^* the ground-truth function, $\boldsymbol{a}_i^*$ the ground-truth arguments, and $y_i \in \{0, 1\}$ the binary label indicating whether the predicted tool call constitutes a hallucination. Given input (q_i, c_i), the LLM $\mathcal{M}$ generates a predicted call $\tilde{f}_i(\boldsymbol{a}_i)$. We extract features from the final layer hidden states of the same forward pass that produces this prediction.

Data Collection and Label Generation We utilize the Glaive Function-Calling dataset (GlaiveAI 2024) from Hugging Face, which comprises multi-domain agent interactions annotated with structured tool usage information and corresponding parameters, covering domains such as personal health, finance, and general utility calculations. We pre-process this dataset by grouping samples based on their designated tool names and normalizing argument structures to ensure consistency in both function and parameter representation. To evaluate tool-specific behaviors and hallucination rates, we develop specialized agents using the Strands framework (Agents 2025) that are programmatically matched to tool names and parameter schemas extracted from the pre-processed samples. These agents are then deployed to receive data samples filtered by tool assignment, enabling precise measurement of agent performance and tool invocation fidelity under controlled experimental conditions.

We collect agent interaction data from various reasoning domains spanning personal health calculations, financial computations, sustainability metrics, and general calculator func-